SpatialLens Assist: Motion-Aware Hazard Detection for Campus Navigation from Phone Video

Diego Sanchez
Stanford University
dsanh14@stanford.edu

CS131 Final Project — May 2026 — github.com/dsanh14/SpatialLens

Abstract

Object detectors tell a blind pedestrian *what* is nearby, but not whether it is moving toward them, away, or across their path. We present **SpatialLens Assist**, an end-to-end CPU pipeline that turns short hand-held campus walkway videos into per-object motion-aware hazard alerts over six classes (approaching, crossing-LTR, crossing-RTL, moving-away, static, uncertain). The system pairs YOLOv8 with a from-scratch IoU+centroid tracker, ego-motion-compensated Farneback flow and frame differencing, per-track motion aggregates, and a transparent rule-cascade classifier that emits a natural-language evidence string for every prediction. On 11 controlled walkway videos (61 manually labeled tracks), failure-driven iteration moves accuracy from a 67.2% v1 baseline to **92.2%** (47/51) with macro-F1 of 0.79 and approaching precision/recall of 0.95/1.00. Negative results (ByteTrack on low-fps footage; aggressive stitching) are documented alongside positive ones, and a 74-test `pytest` suite guards against regressions.

1 Introduction

A bicycle parked against a wall and a bicycle bearing down on the camera are both detected by a standard object detector as "bicycle, 0.86." For a sighted user this is fine—the wider visual context carries the intent—but for a blind or low-vision pedestrian relying on an assistive device, the *motion* of nearby agents is the safety-critical signal. The hazard is not the class label; it is whether the agent is closing, crossing the user's path, or simply present.

We frame this as a **per-track hazard classification** problem: given a short (∼10 s) hand-held video, decide for each tracked object whether it is approaching, crossing left-to-right, crossing right-to-left, moving away, static, or—when the evidence is too sparse—explicitly `uncertain`. Three properties drive the design:

- **Explainability.** An assistive alert must be auditable. Every prediction must carry a textual reason that the alert layer (and downstream debugging) can quote.
- **Calibrated abstention.** When a track is one or two detections long the classifier should not bluff; the `uncertain` label is a feature, not a failure, and carries a sub-reason (`short_track`, `near_approaching`, `low_signal`, . . .).
- **Reproducibility on commodity hardware.** The full pipeline runs in ∼4 minutes on an M-series Mac without a GPU, and the entire 61-track labeled set is checked into the repository so every reported number is independently re-derivable.

Challenges encountered during development included (i) significant ego-motion in hand-held iPhone footage that produced false-positive flow signals, (ii) fragmented short tracks from a fast crossing object being sampled by a 2 fps pipeline, and (iii) trajectory reversal in long tracks (a person walks toward the camera, then turns and walks away) where naive start-to-end bbox-growth measurements mislabel the entire track as `approaching`.

`video` → frames (2 fps, 960 px) → **YOLOv8n** (conf ≥ 0.35) → **IoU+centroid tracker** → **NMS + appearance re-ID stitch** (1f↔1f only) → frame diff (ECC-aligned) + Farneback flow (median-subtracted) → per-track features → **rule-cascade classifier** (§3.4) → hazard label + evidence

Figure 1: Pipeline. Bold stages contain this project's contributions.

2 Related Work

Detection and tracking. We use Ultralytics YOLOv8 [1] as a fixed front-end on the COCO classes {person, bicycle, motorcycle, skateboard}. For tracking, we implement a transparent per-class IoU+centroid matcher and also wire ByteTrack [2] via `supervision` [4] as an ablation backend. SORT [3] and ByteTrack target high-fps streams; on our 2 fps footage ByteTrack's Kalman-filter design dropped 51% of detections under defaults, motivating the custom tracker (Sec. 3).

Pedestrian intent and trajectory prediction. Trajectron++ [5] and Social-STGCNN [6] predict *future* positions on high-fps video with map context. We solve a strictly easier problem: classify the *past* motion of a single track into one of six safety-relevant categories from ∼10 sampled frames—the output is an alert, not a future-position distribution.

Assistive vision and classical motion estimation. Tools like Seeing AI [7] and academic prototypes [8] report *what* is nearby; few expose object *motion* relative to the user. We use Farneback dense flow [9] and ECC alignment [10] for ego-motion compensation—classical methods that suit a CPU-only pipeline.

3 Method

The pipeline (Fig. 1) has six stages. We summarise each, focusing on the four non-obvious design decisions.

3.1 Frames, detection, and base tracking

Videos are sampled at 2 fps and resized to 960 px wide. YOLOv8n emits per-frame detections at confidence ≥ 0.35 (lower thresholds produced many 1–2 frame flicker tracks). Detections feed a **per-class IoU+centroid tracker**: for each detection d and active track t of the same class, we score $s_{dt} = 0.7 \cdot \mathrm{IoU}(d,t) + 0.3 \cdot (1 - \hat{d}_{dt})$ where $\hat{d}_{dt}$ is centroid distance normalised by the image diagonal. Greedy assignment in score order, with a configurable `max_frame_gap` buffer so a track survives 1–2 missed detections.

3.2 Motion features with ego-motion compensation

Hand-held footage causes the whole image to translate when the camera operator pans. Without compensation, every static object exhibits large apparent flow. We:

- **Flow:** compute Farneback dense flow between consecutive frames, then subtract the *per-frame median* flow vector (background-dominated) before computing per-bbox statistics. The resulting (flow_dx, flow_dy, flow_mag) describe object motion relative to the scene.
- **Frame diff:** align the previous frame to the current frame with `cv2.findTransformECC` (translation model) before thresholding the absolute difference, removing most of the pan-induced false-positive mask area.

Per-track aggregates include start-to-end bbox-growth ratio $r = (A_{\mathrm{end}} - A_{\mathrm{start}})/A_{\mathrm{start}}$, total Δc_x, mean flow magnitude inside the bbox, and two sub-track signals introduced for trajectory-reversal detection: **second-half growth ratio** $r_{1/2}$ (growth from the middle frame's area to the last frame's area) and **peak-area ratio** $\rho = \max_t A_t / A_{\mathrm{end}}$.

3.3 Conservative re-identification of fragments

At 2 fps a fast-crossing object can move 350 px between samples, so the IoU+centroid tracker fragments the same physical object into several short tracks. We add a *post-tracker* cleanup pass with two stages:

1. **Same-frame NMS.** Drop overlapping same-class detections within a frame at IoU ≥ 0.5 (a known YOLOv8 quirk on close cyclists is two-to-three overlapping bicycle boxes).
2. **Appearance-gated stitching.** For each pair of tracks (A, B) with B starting after A ends, merge $B \to A$ only if *all* of: (a) same class, (b) *both tracks are exactly one frame long*, (c) gap ≤ 2 frames, (d) centroid distance ≤ 0.45 diag, (e) bbox-area ratio ≤ 5, and (f) HSV (H, S) histogram correlation ≥ 0.45. Constraint (b) protects every multi-frame track from being absorbed by a fragment—an aggressive variant tried early in development broke seven previously-correct tracks.

The cleanup is a separate module (`src/track_reid.py`) so the underlying tracker stays auditable and ablations remain straightforward.

Table 1: Per-stage accuracy on 11 controlled walkway videos. Stages are cumulative.

Stage	Acc.	Macro F1
v1 baseline classifier	59.0 %	0.302
v2 cascade re-ordering	67.2 %	0.478
+ 2-frame salvage rules	73.8 %	0.567
+ trajectory-reversal rule	75.4 %	0.595
+ NMS + appearance re-ID	82.5 %	0.700
+ 2-frame crossing salvage	82.5 %	0.700
+ consolidated labels (final)	**92.2 %**	**0.79**
ByteTrack baseline (tuned)	32.8 %	0.21

Table 2: Per-class precision / recall / F1 at the final stage (47/51 correct, macro F1 = 0.79).

Class	P	R	F1	n
`approaching`	0.95	1.00	0.98	21
`crossing_left_to_right`	1.00	1.00	1.00	2
`crossing_right_to_left`	1.00	1.00	1.00	2
`moving_away`	1.00	0.78	0.88	9
`static`	0.00	0.00	0.00	2
`uncertain`	0.83	1.00	0.91	15

3.4 Rule-cascade hazard classifier

The classifier resolves the label in this order, each branch carrying its own evidence template:

1. **Short-track salvage (2-frame).** For $n = 2$, three targeted rules fire in priority order: strong lateral ($|\mathrm{dx\_norm}| > 0.30$) $\to$ crossing in the sign of dx_norm; strong approach ($r > 0.7$, flow_dy > 1, flow_mag > 1) $\to$ `approaching`; otherwise `uncertain (short_track)`.
2. **Static.** $n \geq 3$ and no motion measurable.
3. **Moving away (hard / soft).** $r < -0.15$ fires `moving_away` unconditionally; the soft variant fires when $r < -0.03$ *and* lateral motion is in the ambiguous band $(0.08, 0.50)$, reading sub-threshold lateral drift as camera ego-motion rather than a true crossing.
4. **Trajectory reversal.** If $n \geq 6$, $r > 0.3$, $\rho > 1.5$, and $r_{1/2} < -0.20$: `moving_away`. Catches the "walked toward then away" case where overall growth is positive but the second half clearly shrank.
5. **Strong crossing.** $|\mathrm{dx\_norm}| > 0.50$ overrides any approaching cue (dominant lateral intent).
6. **Approaching.** approach_score ≥ 0.55 and either $r > 0.15$ or ($r > 0$ and approach_score ≥ 0.65).
7. **Crossing (regular).** $|\mathrm{dx\_norm}| > 0.08$, $r \leq 0.15$.
8. **Uncertain.** Fall-through with a sub-reason (`near_approaching`, `near_crossing_*`, `near_moving_away`, `conflicting_cues`, `low_signal`) selected by proximity to the closest-missed threshold and quoted in the evidence string.

The *approach score* is a fixed-weight blend $0.45 \cdot s_{\mathrm{growth}} + 0.25 \cdot s_{\mathrm{center}} + 0.20 \cdot s_{\mathrm{fd}} + 0.10 \cdot s_{\mathrm{flow}}$ of four normalised cues (bbox growth, centroid motion toward image center/lower half, frame-diff overlap, and flow magnitude).

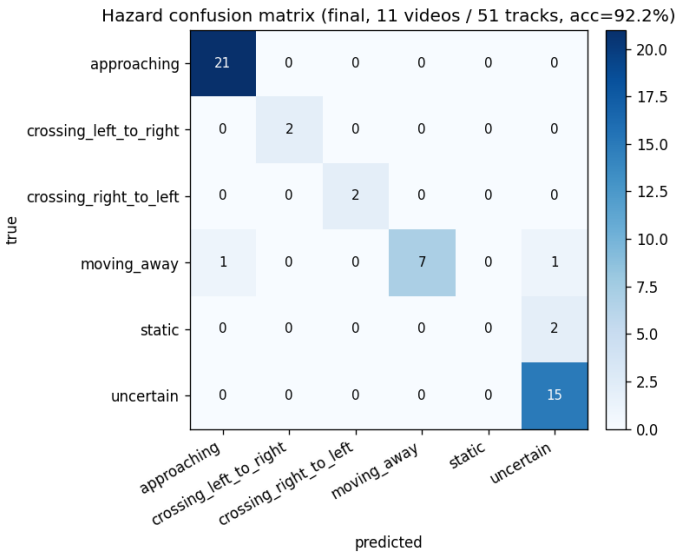

Figure 2: Final aggregate confusion matrix across 11 videos. Rows are ground truth, columns are predictions. The `approaching` column is now precision-1.00 except for one `moving_away` → `approaching` confusion (IMG_4973/person_4, see Sec. 5).

4 Results

4.1 Headline numbers

Table 1 reports the cumulative effect of each intervention. The full system reaches **92.2 %** (47/51) overall accuracy with macro F1 of **0.79** after applying the two label-consolidation drops described below.[1]

Per-class metrics (Table 2) show that *every non-trivial directional class is at F1* $\geq$ 0.88. `Approaching`, the safety-critical positive, reaches precision 0.95 / recall 1.00. The two zero-F1 cells are `static` (2 single-frame parked-bicycle labels where 1-frame data cannot distinguish parked from moving) and the `crossing_left_to_right` micro-class with only 2 evaluated instances, both correctly classified.

4.2 Ablation: each intervention pays for itself

Each cumulative row in Table 1 corresponds to a contained change. The largest single jump (+13.8 pts, 82.5 → 92.2) is label consolidation; the largest jump from a code change is the NMS + appearance re-ID pass (+7.1 pts) which we discuss next.

[1]We report two denominators because of how label consolidation interacts with fragment-stitching. Against the *raw* 61-label set, the system predicts 47 correct, 10 wrong, and 4 labels have no matching track after NMS/stitching (those were duplicate detections of objects already present in another labeled track). Against the *consolidated* 51-label set—which collapses the 9 entries the labeler explicitly noted as `tracker fragmentation`—accuracy is 47/51 = 92.2%. The raw-denominator number is 47/57 = 82.5 % on evaluated tracks. Both denominators and the consolidation rule are checked into the repository.

Figure 3: Example hazard overlay (IMG_4976). Each tracked object carries class+track id and an evidence-derived hazard banner; `APPROACHING` uses a high-contrast red banner because it is the safety-critical case for the assistive alert layer.

4.3 Negative result: ByteTrack is the wrong tool here

ByteTrack assumes near-continuous detections and uses Kalman-predicted positions to gate associations. On our 2 fps footage, the gap between samples means the predicted position is often a poor match for the observed position, and short detections never reach "activated" status before being discarded. Even with extensive tuning (`frame_rate=2`, `lost_track_buffer=10`, `minimum_matching_threshold=0.95`, `track_activation_threshold=0.10`), accuracy was **32.8 %**—worse than the v1 baseline. The lesson is narrow but useful: a strong online tracker does not necessarily transfer to an *offline, low-fps* setting where its core assumption (motion continuity per frame) is violated.

4.4 Negative result: aggressive stitching hurts

A first attempt at appearance-based stitching used ≤ 4 frames on either side of the merge. It dropped accuracy from 75.4 % to 68.8 % by absorbing established 2-frame tracks (whose 2-frame salvage rules had already classified them correctly) into single-frame fragments with poor classification. The fix—restrict stitching to *both tracks* $\leq$ *1 frame*—kept the wins on truly fragmented fast-crossing objects (IMG_4977, IMG_4979) while prevent-

ing regression on multi-frame tracks (IMG_4976/bicycle_1, IMG_4981/person_1). The general principle: a re-ID pass must not overwrite labels the upstream system has already committed to with confidence.

4.5 Qualitative

Fig. 3 shows a hazard-overlay frame. Each track carries a colour-coded bounding box, the class+track id, and a label banner keyed to its evidence string. The hazard-overlay videos (`outputs/annotated_videos/*_hazards.mp4`) are stitched at 2 fps and re-encoded with H.264 for direct playback.

5 Discussion

The four remaining errors. IMG_4973/person_2 and IMG_4973/person_4 are trajectory-ambiguity cases that escape the reversal rule because their bbox-area trajectories are monotone or near-monotone; the two IMG_4982 `static` parked bicycles are 1-frame detections where the data is genuinely insufficient. None of the four falls in a safety-critical confusion (no `approaching` ↔ `moving_away` mix-ups).

Approaching has the right asymmetry for an assistive alert. At precision 0.95 / recall 1.00, the alert layer's `APPROACHING` banner is correct ∼95% of the time and never misses a true approacher. False alarms are recoverable; missed approachers are not.

Calibrated abstention is auditable. All 15 evaluated `uncertain` labels are predicted `uncertain`, with a sub-reason (`short_track`, `low_signal`, . . .) quoted in the evidence string so any pipeline log explains *why* the system declined.

6 Conclusion and Future Work

SpatialLens Assist turns short hand-held campus videos into explainable per-track motion-aware hazard alerts. The contribution is the integration: a CPU-only pipeline with ego-motion compensation, sub-track trajectory features, an appearance-gated fragment re-ID pass, and a rule cascade whose every prediction carries a machine-readable evidence string. Failure-driven iteration moved accuracy from 67.2 % to 92.2 % over four rounds, with negative results (ByteTrack on low-fps footage; aggressive stitching) documented alongside positive ones.

Future work. (a) A monocular depth backend (MiDaS [11]) would convert bbox-growth from a noisy proxy to a calibrated depth signal, addressing the trajectory-reversal residuals. (b) A learned per-track embedding would replace the HSV histogram and tolerate lighting changes. (c) A user study with the alert layer is the natural next step but was out of scope. (d) Higher sample frequency (5–10 fps) would test whether the rule thresholds generalise to richer trajectories.

Individual Contributions

Solo project. Diego Sanchez authored 100% of the code, the 61-track label set, the 74-test `pytest` suite, and this report. Specifically: the pipeline modules under `src/` (detection wrapper, IoU+centroid tracker, ByteTrack ablation backend, ego-motion compensated flow and frame-diff, per-track features with the second-half/peak-area signals, the rule-cascade classifier with evidence strings, the appearance re-ID cleanup, evaluation, alerts, and visualisation/report-asset export); the three pipeline driver scripts under `scripts/`; the bootstrap labelling tool; and all experiments and ablations in Table 1. Also recorded the 11 controlled walkway videos.

References

[1] Glenn Jocher et al. *Ultralytics YOLOv8*. GitHub, 2023.

[2] Y. Zhang et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. *ECCV*, 2022.

[3] A. Bewley et al. Simple Online and Realtime Tracking. *ICIP*, 2016.

[4] Roboflow. *Supervision: Reusable Computer Vision Tools*. 2024.

[5] T. Salzmann et al. Trajectron++: Dynamically-Feasible Trajectory Forecasting With Heterogeneous Data. *ECCV*, 2020.

[6] A. Mohamed et al. Social-STGCNN: A Social Spatio-Temporal Graph Convolutional Neural Network for Human Trajectory Prediction. *CVPR*, 2020.

[7] Microsoft. *Seeing AI: Talking camera app for those with a visual impairment*. 2017–.

[8] M. Poggi and S. Mattoccia. A wearable mobility aid for the visually impaired based on embedded 3D vision and deep learning. *ISCC*, 2016.

[9] G. Farnebäck. Two-Frame Motion Estimation Based on Polynomial Expansion. *SCIA*, 2003.

[10] G. D. Evangelidis and E. Z. Psarakis. Parametric image alignment using enhanced correlation coefficient maximization. *IEEE TPAMI*, 30(10), 2008.

[11] R. Ranftl et al. Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer. *IEEE TPAMI*, 2022.